

Hands-on Gurobipy

Autor: unknown

May 17, 2026

Convenciones: I, J, K conjuntos de índices; i, j, k índices; a, b, c parámetros; M constante Big-M; m = instancia de `gb.Model()`; x, y, z variables `gurobipy`.

Índice de contenidos

§1 Setup del modelo

- 1.1 Crear modelo e importar librerías
- 1.2 Variable continua con cotas
- 1.3 Variable entera
- 1.4 Variable binaria
- 1.5 Variables indexadas — Loops
- 1.6 Variables indexadas — dict comp.
- 1.7 Variables indexadas — addVars

§2 Restricciones básicas

- 2.1 Igualdad
- 2.2 Desigualdad $\leq$ ponderada
- 2.3 Desigualdad $\geq$ con filtro
- 2.4 Restricciones indexadas — Loops
- 2.5 Restricciones indexadas — addConstrs
- 2.6 Rango (restricción doble)

§3 Sumas y quicksum

- 3.1 quicksum vs sum de Python
- 3.2 Objetivo lineal
- 3.3 Suma doble (multi-índice)
- 3.4 Objetivo con suma doble

§4 Variables binarias y Big-M

- 4.1 Activación con Big-M

- 4.2 Equivalencia $y_i = 1 \Leftrightarrow x_i > 0$

- 4.3 addGenConstrIndicator

§5 Restricciones multi-índice

- 5.1 Para todo par (i, j)
- 5.2 Suma por fila y columna
- 5.3 Doble suma con filtro

§6 Restricciones de red

- 6.1 Conservación de flujo
- 6.2 Capacidad de arco binaria
- 6.3 Asignación: uno activo

§7 Linealización

- 7.1 Valor absoluto $t = |x|$
- 7.2 Máximo $t = \max_i x_i$
- 7.3 Mínimo $t = \min_i x_i$

§8 Avanzado

- 8.1 Parámetros y lectura de resultados

§9 Post-optimización

- 9.1 Precios sombra (dual values)
- 9.2 Holguras (slack/surplus)
- 9.3 Costos reducidos
- 9.4 Ranging del RHS
- 9.5 Ranging coef. objetivo

Sección 1: Setup del modelo

1.1 Crear modelo e importar librerías

$\min f(x)$
 modelo vacío,
 sin variables
 ni restricciones

```
import gurobipy as gb
from gurobipy import GRB, quicksum

m = gb.Model("mi_modelo")      # instancia de modelo
# Opcional para controlar la salida y el tiempo de
# optimización
m.setParam("OutputFlag", 0)    # silenciar salida en
                                # consola
m.setParam("TimeLimit", 300)   # limite de tiempo en
                                # segundos
```

1.2 Variable continua con cotas

$$x \in [lb, ub] \subset \mathbb{R}$$

Sin `ub`, la variable es ilimitada superiormente (`GRB.INFINITY`). `name` aparece en logs y en archivos `.lp` para debug.

```
x = m.addVar(
    vtype=GRB.CONTINUOUS,
    lb=0,          # cota inferior (default: 0)
    ub=100,        # cota superior (default: GRB.INFINITY)
    name="x"
)
```

1.3 Variable entera

$$z \in \mathbb{Z}_{\geq 0}$$

Se puede combinar con `ub` para limitar el rango entero, ejemplo `ub=50`.

```
z = m.addVar(
    vtype=GRB.INTEGER,
    lb=0,          # cota inferior (default: 0)
    ub=GRB.INFINITY, # opcional, es el default para
                    # enteras
    name="z"
)
```

1.4 Variable binaria (caso particular de entera con $lb = 0$ y $ub = 1$)

$$y \in \{0, 1\}$$

Es equivalente a declarar la variable como entera y limitar su rango a $[0, 1]$. Si se declara como binaria, no es necesario declarar cotas.

```
y = m.addVar(vtype=GRB.BINARY, name="y")
```

1.5 Variables indexadas — Loops anidados

$$x_{ij} \in [0, 1]$$

$$\forall i \in I, j \in J$$

El doble loop es la forma más directa de crear variables indexadas. Sin embargo, para modelos con muchos índices, el código se vuelve verboso y propenso a errores.

```
I = ["A", "B", "C"]
J = [1, 2, 3]
X = {}                                # diccionario vacio para guardar
    las variables
for i in I:                            # para cada i en I
    for j in J:                        # para cada j en J
        X[i, j] = m.addVar(
            vtype=GRB.CONTINUOUS,
            lb=0, ub=1,
            name=f"X_{i}_{j}"
        )
```

1.6 Variables indexadas — dict comprehension

$$x_{ij} \in [0, 1]$$

$$\forall i \in I, j \in J$$

Clave tupla: $X["A", 1] \equiv X[("A", 1)]$. El dict comprehension es una forma compacta de crear variables indexadas sin escribir bucles anidados. Ideal para modelos con muchos índices.

```
I = ["A", "B", "C"]
J = [1, 2, 3]

X = {
    (i, j): m.addVar(vtype=GRB.CONTINUOUS,
                     lb=0, ub=1,
                     name=f"X_{i}_{j}")
    for i in I
    for j in J
}
```

1.7 Variables indexadas — addVars y multidict

x_{ij}, c_{ij}, u_{ij}
desde datos estructurados

multidict devuelve claves + dicts en una sola línea, es decir, *arcs* = lista de claves.

addVars devuelve *tupledict* con método *.sum("A", "*")* para sumas parciales sin escribir *quicksum*. Por cada valor en *arcs*, se crea una variable con el mismo nombre, ejemplo $x["A", "B"]$.

```
arcs, cost, cap = gb.multidict({
    ("A", "B"): [2.5, 10],
    ("A", "C"): [1.8, 5],
    ("B", "C"): [3.0, 8],
})

# addVars devuelve tupledict
x = m.addVars(arcs, vtype=GRB.CONTINUOUS,
              lb=0, name="x")

# suma parcial sobre todo j con origen "A"
expr = x.sum("A", "*")
```

Sección 2: Restricciones básicas

2.1 Igualdad

$$\sum_{i \in I} x_i = b$$

La igualdad se modela con `==` en *addConstr*. En modelos grandes, es recomendable usar *quicksum* para construir la expresión lineal de manera eficiente.

```
b = 1.0
m.addConstr(
    quicksum(x[i] for i in I) == b,
    name="igualdad"
)
```

2.2 Desigualdad $\leq$ con suma ponderada por a_i

$$\sum_{i \in I} a_i x_i \leq b$$

```
a = {"A": 3, "B": 2, "C": 5}
b = 10

m.addConstr(
    quicksum(a[i] * x[i] for i in I) <= b,
    name="capacidad"
)
```

2.3 Desigualdad $\geq$ — con restriccion de elementos en I

$$\sum_{i \in I \setminus \{B\}} x_i \geq 1$$

```
m.addConstr(
    quicksum(x[i] for i in I if i != "B") >= 1,
    name="cobertura"
)
```

2.4 Restricciones indexadas con elementos excluidos de I — Loops

$$\sum_{\substack{j \in J \\ i \neq j}} x_{ij} \leq u_i, \quad \forall i \in I$$

```
u = {"A": 2, "B": 3, "C": 1}
for i in I:
    m.addConstr(
        quicksum(X[i, j] for j in J if j != i) <= u[i],
        name=f"capacity_{i}"
    )
```

2.5 Restricciones indexadas — Vectorizadas con addConstrs y quicksum

$$\sum_{\substack{j \in J \\ i \neq j}} x_{ij} \leq u_i, \quad \forall i \in I$$

addConstrs permite crear múltiples restricciones de manera vectorizada, evitando bucles explícitos.

```
u = {"A": 2, "B": 3, "C": 1}
m.addConstrs(
    (quicksum(X[i, j] for j in J if j != i) <= u[i]
     for i in I),
    name="capacity"
) # crea: capacity[A], capacity[B], capacity[C]
```

2.6 Rango (restricción doble)

$$L \leq \sum_{i \in I} x_i \leq U$$

addRange crea 1 fila interna. Dos addConstr crean 2 filas pero son más legibles en el log.

```
expr = quicksum(x[i] for i in I)

# opcion 1: una sola restriccion interna
m.addRange(expr, 2, 5, name="rango")

# opcion 2: equivalente, dos restricciones
m.addConstr(expr >= 2, name="rango_lb")
m.addConstr(expr <= 5, name="rango_ub")
```

Sección 3: Sumas y quicksum

3.1 quicksum vs sum de Python

$$\sum_{i \in I} c_i x_i$$

quicksum construye la LinExpr sin objetos intermedios. Con $|I| = 10^4$, `sum()` puede ser hasta 100× más lento porque crea expresiones temporales en cada $\dagger$.

```
c = {"A": 2.0, "B": 3.5, "C": 1.0}

# CORRECTO: eficiente con gurobipy
expr = quicksum(c[i] * x[i] for i in I)

# EVITAR en modelos grandes (crea LinExpr temporales)
expr_lenta = sum(c[i] * x[i] for i in I)
```

3.2 Objetivo lineal

$$\min_x \sum_{i \in I} c_i x_i$$

```
m.setObjective(
    quicksum(c[i] * x[i] for i in I),
    GRB.MINIMIZE # o GRB.MAXIMIZE
)
```

3.3 Suma doble (multi-índice)

$$\sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij}$$

$cost[i, j] \equiv cost[(i, j)]$.

```
cost = {("A", 1): 2.0, ("A", 2): 1.5,
        ("B", 1): 3.0, ("B", 2): 2.5}

obj = quicksum(
    cost[i, j] * x[i, j]
    for i in I
    for j in J
)
m.setObjective(obj, GRB.MINIMIZE)
```

3.4 Objetivos con suma doble (multi-índice)

$$\sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} + \sum_{i \in I} d_i y_i$$

Se pueden combinar sumas de diferentes índices en un mismo objetivo. Solo es necesario construir la expresión completa con `quicksum` antes de llamar a `setObjective`.

```
cost = {("A",1): 2.0, ("A",2): 1.5,
        ("B",1): 3.0, ("B",2): 2.5}
d = {"A": 1.0, "B": 0.5, "C": 2.0}
obj = quicksum(
    cost[i, j] * x[i, j]
    for i in I
    for j in J
) + quicksum(
    d[i] * y[i]
    for i in I
)
m.setObjective(obj, GRB.MINIMIZE)
```

Sección 4: Variables binarias y Big-M

4.1 Activación: $y_i = 1 \Rightarrow x_i \leq M$ y $y_i = 0 \Rightarrow x_i = 0$

$$x_i \leq M \cdot y_i$$

$$\forall i \in I$$

M debe ser la cota superior ajustada de x_i .
Ojo: un M grande debilita la relación LP y enlentece el Branch & Bound.

```
M = 100 # cota superior real de x[i]

for i in I:
    m.addConstr(
        x[i] <= M * y[i],
        name=f"bigM_{i}"
    )
# y[i]=0 -> x[i]=0 ; y[i]=1 -> x[i] libre hasta M
```

4.2 Equivalencia: $y_i = 1 \Leftrightarrow x_i > 0$

$$x_i \leq M \cdot y_i$$

$$x_i \geq \varepsilon \cdot y_i$$

$$\forall i \in I$$

ε es un umbral pequeño (10^{-3}) que fuerza $y_i = 1$ cuando $x_i > 0$. Solo válido si x_i es continua.

```
eps = 1e-3

for i in I:
    m.addConstr(x[i] <= M * y[i],
                name=f"bigM_ub_{i}")
    m.addConstr(x[i] >= eps * y[i],
                name=f"bigM_lb_{i}")
```

4.3 addGenConstrIndicator — implicación limpia

$$y_i = 1 \Rightarrow \sum_{j \in J} x_{ij} \leq C_i \\ \forall i \in I$$

Más legible que Big-M. Gurobi lo convierte internamente a Big-M durante el preproceso. No garantiza ventaja sobre un Big-M bien formulado.

```
C = {"A": 5, "B": 3}

for i in I:
    m.addGenConstrIndicator(
        y[i],      # binaria "switch"
        True,      # valor del switch: True=1, False=0
        quicksum(X[i, j] for j in J) <= C[i],
        name=f"indicator_{i}"
    )
```

Sección 5: Restricciones multi-índice

5.1 Para todo par (i, j)

$$x_{ij} \leq y_i \\ \forall i \in I, j \in J$$

```
for i in I:
    for j in J:
        m.addConstr(
            X[i, j] <= y[i],
            name=f"link_{i}_{j}"
        )
```

5.2 Suma por fila y por columna

$$\sum_{j \in J} x_{ij} = 1 \quad \forall i \\ \sum_{i \in I} x_{ij} \leq 1 \quad \forall j$$

```
# restriccion por fila (para cada i)
for i in I:
    m.addConstr(
        quicksum(X[i, j] for j in J) == 1,
        name=f"fila_{i}"
    )

# restriccion por columna (para cada j)
for j in J:
    m.addConstr(
        quicksum(X[i, j] for i in I) <= 1,
        name=f"col_{j}"
    )
```

5.3 Doble suma con filtro

$$\sum_{k \in K} f_{ikt} x_{ikt} \\ \leq \sum_{\substack{q \in Q \\ c_{iq} > 0}} c_{iq} z_{iq} \\ \forall i \in I, t \in T$$

El filtro *if cap[i, q] > 0* evita términos nulos que engordan el modelo sin aportar restricción real.

```
for t in T:
    for i in I:
        m.addConstr(
            quicksum(
                f[i, k, t] * X[i, k, t]
                for k in K)
            <=
            quicksum(
                cap[i, q] * Z[i, q, t]
                for q in Q
                if cap[i, q] > 0), # filtrar ceros
            name=f"cap_{i}_{t}")
```

Sección 6: Restricciones de red

6.1 Conservación de flujo

$$\sum_{j: (i,j) \in A} x_{ij} \\ - \sum_{j: (j,i) \in A} x_{ji} = b_i \\ \forall i \in V$$

$b_i < 0$: fuente; $b_i > 0$: sumidero; $b_i = 0$: tránsito. Los arcos se filtran dinámicamente desde la lista *arcs*.

```
nodes = ["s", "1", "2", "t"]
arcs = [("s","1"), ("s","2"),
        ("1","t"), ("2","t")]
b = {"s": -1, "1": 0, "2": 0, "t": 1}

x = {a: m.addVar(lb=0, name=f"x_{a}")
      for a in arcs}

for i in nodes:
    salida = quicksum(x[u,v] for (u,v) in arcs
                      if u == i)
    entrada = quicksum(x[u,v] for (u,v) in arcs
                      if v == i)
    m.addConstr(salida - entrada == b[i],
                name=f"flujo_{i}")
```


6.2 Capacidad de arco con apertura binaria

$$0 \leq x_{ij} \leq u_{ij} \cdot y_{ij}$$

$$\forall (i,j) \in A$$

```
u = {"s","1"): 10, ("s","2"): 5,
      ("1","t"): 8, ("2","t"): 6}

y = {a: m.addVar(vtype=GRB.BINARY,
                  name=f"y_{a}")
      for a in arcs}

for a in arcs:
    m.addConstr(x[a] <= u[a] * y[a],
                 name=f"cap_{a}")
```

6.3 Asignación: exactamente uno activo

$$\sum_{i \in I} y_i = 1$$

```
m.addConstr(
    quicksum(y[i] for i in I) == 1,
    name="exactamente_uno"
)
```

Sección 7: Linealización de no-linealidades

7.1 Valor absoluto: $t = |x|$

$$t \geq x$$

$$t \geq -x$$

$$t \geq 0$$

$$\Leftrightarrow t = |x|$$

(si se minimiza t)

Las dos formas son equivalentes.
`addGenConstrAbs` es más conciso;
 Gurobi lo convierte a las dos desigualdades durante el preproceso.

```
x = m.addVar(lb=-GRB.INFINITY, name="x")
t = m.addVar(lb=0, name="t_abs")

# Forma manual (dos restricciones lineales)
m.addConstr(t >= x, name="abs_pos")
m.addConstr(t >= -x, name="abs_neg")

# Forma directa con restriccion generalizada
m.addGenConstrAbs(t, x, name="abs_x")
# minimizar t en objetivo -> t = |x|
```

7.2 Máximo: $t = \max_i x_i$

$$t = \max_{i \in I} x_i$$

Solo cotas $t \geq x_i$ es suficiente si t entra en el objetivo con coeficiente > 0 (minimización). Para igualdad exacta en otros contextos usar `addGenConstrMax`.

```
t_max = m.addVar(lb=-GRB.INFINITY, name="t_max")

# Opcion 1: solo cotas inferiores (si minimizando t)
for i in I:
    m.addConstr(t_max >= x[i],
                name=f"max_lb_{i}")

# Opcion 2: igualdad exacta garantizada
m.addGenConstrMax(
    t_max,
    [x[i] for i in I],
    constant=-GRB.INFINITY, # sin constante adicional
    name="max_x"
)
```

7.3 Mínimo: $t = \min_i x_i$

$$t = \min_{i \in I} x_i$$

```
t_min = m.addVar(lb=-GRB.INFINITY, name="t_min")

m.addGenConstrMin(
    t_min,
    [x[i] for i in I],
    constant=GRB.INFINITY, # sin constante adicional
    name="min_x"
)
```

Sección 8: Avanzado

8.1 Parámetros del solver y lectura de resultados

Configuración
del solver
Lectura de
resultados tras
`m.optimize()`

```
m.setParam("MIPGap", 0.01) # gap optimalidad 1%
m.setParam("TimeLimit", 3600) # 1 hora maximo
m.setParam("Threads", 4) # hilos en paralelo
m.setParam("MIPFocus", 1) # buscar factible rapido
m.setParam("Presolve", 2) # presolve agresivo
m.setParam("OutputFlag", 1) # activar log en consola

m.optimize()

# Leer resultados
if m.Status == GRB.OPTIMAL:
    print(f"ObjVal: {m.ObjVal:.4f}")
    print(f"ObjBound: {m.ObjBound:.4f}")
    print(f"MIPGap: {100*m.MIPGap:.2f}%")
    print(f"Runtime: {m.Runtime:.1f}s")
    sol = {i: x[i].X for i in I} # .X = valor optimo
```

Sección 9: Análisis de sensibilidad post-optimización

9.1 Precios sombra (*dual values*)

$$\lambda_i = \frac{\partial z^*}{\partial b_i}$$

$$\Delta z^* \approx \lambda_i \cdot \Delta b_i$$

$\forall i \in \text{restricciones}$

Solo LP (no MIP directo). En min:
 $\lambda_i > 0 \Rightarrow$ relajar b_i reduce z^* . Re-
 stricción "cara": activa con $\lambda_i \neq 0$.

```
m.optimize() # LP requerido para analisis de sensibilidad

for c in m.getConstrs():
    print(f"{c.ConstrName:15s}  Pi = {c.Pi:+.4f}")

# Pi = -2.5 (min): relajar b_i en 1 reduce z* en 2.5
c_cap = m.getConstrByName("capacidad")
print(f"precio sombra: {c_cap.Pi:.4f}")
```

9.2 Holguras (*slack / surplus*)

$$s_i = b_i - \sum_j a_{ij} x_j^*$$

$s_i = 0$: restricción activa

$s_i > 0$: restricción inactiva

Restricción inactiva $\Rightarrow \lambda_i = 0$. Para
 $\geq$: $s_i = \text{lhs} - b_i$. Disponible en LP
 y MIP.

```
for c in m.getConstrs():
    activa = abs(c.Slack) < 1e-6
    estado = "activa" if activa else "inactiva"
    print(f"{c.ConstrName}: slack={c.Slack:.4f} [{estado}]")
```

9.3 Costos reducidos (*reduced costs*)

$$\bar{c}_j = c_j - \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{a}_j$$

Var. básica: $\bar{c}_j = 0$

No básica (min): $\bar{c}_j \geq 0$

$|\bar{c}_j|$ = cuánto debe cambiar c_j para
 que x_j entre a la base. Si $x_j^* > 0$,
 entonces $RC = 0$.

```
for v in m.getVars():
    estado = "basica" if abs(v.RC) < 1e-6 else "no basica"
    print(
        f"{v.VarName:10s}  x*={v.X:.4f}"
        f"  RC={v.RC:+.4f} [{estado}]"
    )
```

9.4 Ranging del RHS

$b_i \in [\text{Low}_i, \text{Up}_i]$
 λ_i constante en ese
 intervalo (misma base)
 Fuera del rango:
 la base cambia

SARHSLow/Up: límites de b_i donde λ_i sigue siendo válido. Si b_i sale del rango, la base cambia.

```
for c in m.getConstrs():
    print(
        f"{c.ConstrName:15s}"
        f"   b_i in [{c.SARHSLow:.3f}, {c.SARHSUp:.3f}]"
        f"   Pi={c.Pi:.4f}"
    )
# [Low, Up]: intervalo donde lambda_i permanece constante
```

9.5 Ranging de coeficientes objetivo

$c_j \in [\text{Low}_j, \text{Up}_j]$
 Base óptima no cambia
 en ese intervalo

Fuera del rango, otra variable puede entrar a la base y x^ cambia. Útil para evaluar robustez frente a cambios en costos.*

```
for v in m.getVars():
    print(
        f"{v.VarName:10s}"
        f"   c_j={v.Obj:.3f}"
        f"   in [{v.SAObjLow:.3f}, {v.SAObjUp:.3f}]"
    )
# SAObjLow/Up: rango de c_j donde la base actual
# sigue siendo optima (misma solucion x*)
```

Referencia: Gurobi Optimizer Reference Manual v10.0